

Arhitectură pentru un plugin Paper care simulează o lume AI cu scenarii multiple

Rezumat executiv

Pentru un plugin Paper care vrea să creeze o „lume vie” cu NPC-uri, economii locale, vreme, memorie și scenarii multiple, soluția robustă nu este să delegi tot comportamentul către un LLM, ci să folosești un **motor hibrid**: reguli locale și behavior trees pentru bucla de gameplay, plus apeluri AI rare și scurte pentru planificare, dialog, generare de obiective, explicarea evenimentelor și variație narativă. Documentația Paper curentă expune exact zonele API de care ai nevoie pentru această direcție: Entity API, Mob Goal API, Entity Pathfinder API, PDC, scheduling, particles și compatibilitate Paper/Folia. În același timp, Paper recomandă explicit ca operațiile lente precum web requests, fișierele și bazele de date să fie mutate în async, cu avertismentul că o parte mare din Bukkit API nu este sigură în thread-uri asincrone dacă accesează sau schimbă starea lumii. ¹

Pe hardware-ul presupus aici — un VPS modest, fără constrângeri specificate de GPU — recomandarea practică este: **LLM-ul nu conduce fiecare NPC în fiecare tick**. În schimb, NPC-urile rulează local la frecvență mare, pe modele de percepție și acțiune discrete, iar AI-ul extern intervine la frecvență mică, pe agregate sau pe evenimente-cheie. Pentru persistență începe cu SQLite, deoarece driverul JDBC este inclus în Paper; dacă lumea trece de la prototip la server cu mulți jucători și acces concurent, mută starea persistentă într-o bază standalone cu pooling, de exemplu prin HikariCP. ²

Pentru API-uri ieftine și rapide, combinația cea mai convingătoare în sursele oficiale este: **Groq** pentru viteză brută de inferență pe modele open, **Gemini 2.5 Flash-Lite** pentru cost mic, free tier și workload-uri cu latență foarte mică, **OpenAI GPT-5.4 mini** pentru integrare generală bună și capabilități mature de tool use, **Claude Haiku 4.5** pentru bucle agentice rapide și caching bun, iar **Mistral Small 4** pentru preț favorabil și plan gratuit de experimentare. Pentru self-hosting, alege între **Ollama** pentru simplitate, **llama.cpp server** pentru footprint mic și compatibilitate OpenAI, **vLLM** pentru throughput mare pe GPU și **LocalAI** pentru compatibilitate multi-provider și control local al datelor. ³

Din punct de vedere al designului, scenariile bune stau pe un spectru între “strong story” și “strong autonomy”: evenimente strict scriptate funcționează excelent pentru momente de showcase, iar simularea emergentă oferă replayability și senzația de lume autonomă. Literatura despre believable agents și interactive drama arată exact această tensiune între control narativ și autonomie, iar literatura despre procedural content generation arată că generarea procedurală este utilă tocmai pentru varietate, scală și conținut greu de reprodus identic. Behavior trees rămân o alegere foarte bună pentru jocuri deoarece s-au impus tocmai ca răspuns la problemele de scalare ale finite state machines. ⁴

Cerințe funcționale și tipuri de scenarii

Ca produs, pluginul ar trebui să fie gândit mai puțin ca „NPC-uri cu chat” și mai mult ca un **simulator de sisteme**. Minimul util este un set de feature-uri care decuplează percepția, decizia, mobilitatea, economia, starea socială și evenimentele globale. Paper oferă ghiduri dedicate pentru goals, pathfinding, task scheduling, PDC, particles și suport Paper/Folia, deci există suport API direct pentru o implementare fără NMS pentru mare parte din arhitectură. ⁵

Domeniu	Ce trebuie să facă pluginul	De ce contează
Agenți AI	NPC-uri cu roluri, nevoi, relații, memorie, inventar, obiective și rutină	Fără stare internă persistentă, „inteligența” pare doar cosmetică
Mobilitate	Pathfinding, evitare de congestie, fallback la teleport controlat, recovery după blocaje	Paper rulează la 20 tps; orice blocaj vizibil degradează realismul și performanța ⁶
Economie	Producție, consum, prețuri locale, penurie, rute comerciale, impozite/sinks	Creează cauzalitate între scenarii și comportamente
Simțuri	Vedere, auz, proximitate, memorie parțială, reputație și zvonuri	Observabilitatea incompletă produce comportament mai plauzibil decât omnisciența totală ⁷
Lume	Vreme, anotimpuri, hazard, resurse regenerabile, biomi, evenimente globale	Dă context extern deciziilor
Persistență	Salvare incrementală pentru agenți, gospodării, loturi, evenimente și cache AI	Fără persistență, lumea „uită” și realismul se rupe
Prezentare	Particles, sunete, delay-uri, micro-animatii, semnalizare vizuală	Paper și Spigot oferă instrumente directe pentru particles și categorii audio ⁸

Tipurile de scenarii merită tratate ca **șabloane operaționale**, nu doar „moduri de joc”. Literatura de interactive drama descrie o axă între autonomie puternică și control narativ puternic. Pentru pluginul tău, asta se traduce în patru familii de scenarii: lume deschisă, evenimente scriptate, AI emergentă și contexte cu resurse limitate. ⁹

Tip de scenariu	Definiție operațională	Când e potrivit	Presiune pe AI	Grad de determinism
Open-world	NPC-urile urmăresc obiective locale și globale, fără final prestabilit	Servere sandbox, survival, roleplay persistent	Medie	Mediu

Tip de scenariu	Definiție operațională	Când e potrivit	Presiune pe AI	Grad de determinism
Scripted events	Quest-uri, festivaluri, războaie, evacuări sau crize pornite de trigger-e clare	Showcase, onboarding, campanii narative	Joasă spre medie	Ridicat
Emergent AI	Comportamentul apare din interacțiunea sistemelor, nu din scenarii scrise manual	Replayability ridicat, storytelling „organic”	Ridicată în design, nu în cost API	Scăzut spre mediu
Resource-limited	Foamete, iarnă, colaps logistic, embargo, lipsă de energie	Dacă vrei bucle economice și decizii tensionate	Medie	Mediu

Pentru un MVP bun, aş construi **două scenarii demonstrabile** şi nu patru simultan: un open-world economic şi un scripted disaster scenario. Dacă acestea funcţionează, apoi activezi emergenţa şi scarcity ca modificatori de sistem, nu ca proiecte separate. Această ordine reduce riscul de a construi prea devreme sisteme „inteligente” imposibil de verificat. Recomandarea este o inferenţă de proiectare, dar este susţinută de faptul că behavior trees şi PCG se scalează bine când comportamentul este modular şi compoziţional, nu monolitic. ¹⁰

Mecanici de bază şi realism perceput

Mecanica de bază ar trebui să fie împărţită în două straturi. Primul este **simularea locală deterministă**, care rulează des şi ieftin: nevoi, foame, oboseală, siguranţă, rutine, pathfinding, crafting, cumpărare, vânzare, reacţii la vreme, reacţii la prezenţa jucătorului. Al doilea este **stratul deliberativ rar**, care rulează mai rar şi poate apela AI extern pentru a transforma starea agregată în planuri, text, priorităţi sau obiective de facţiune. Paper avertizează direct că task-urile lente trebuie mutate în async şi că metodele care accesează sau modifică world state nu sunt sigure off-thread, deci această separare nu este doar elegantă, ci necesară tehnic. ⁶

În practică, mecanicile importante sunt:

- **Agenţi:** fiecare NPC are rol, temperament, buget, memorie episodică scurtă şi ţinte pe termen scurt/lung.
- **Pathfinding:** foloseşte Pathfinder API/goal-uri existente pentru mişcare, plus fallback-uri dacă un agent e blocat.
- **Decision-making:** behavior tree sau utility AI pentru acţiuni locale; LLM doar pentru replanificare rară.
- **Economie:** bunuri, joburi, preţuri locale, depozite, penurie, taxe, pierderi logistice.
- **Vreme şi hazard:** vremea schimbă eficienţa muncii, mobilitatea, riscul şi cererea.
- **Persistenţă:** starea se salvează incremental şi se reîncarcă fără „reset narativ”.
- **Input senzorial:** agenţii nu cunosc tot; văd, aud, deduc şi uită.
- **Învăţare/adaptare:** modifiţi ponderi şi preferinţe, nu reantrenezi LLM-ul în timp real.
- **Aleatoriu vs determinism:** seed global plus zgomot local controlat, ca să poţi reproduce bug-uri fără a distruge varietatea.

Behavior trees sunt de obicei alegerea cea mai sănătoasă pentru stratul de acțiune deoarece sunt modulare, reutilizabile și s-au impus în jocuri tocmai pentru că finite state machines scalează slab când crește complexitatea. Pentru „believability”, literatura clasică subliniază că agenții par mai credibili când au execuție reactivă orientată spre scopuri și când se adaptează incremental, nu doar execută scripturi rigide.

11

Partea cea mai greu de reprodus de către alte pluginuri nu este „apelul la AI”, ci **modul în care ascunzi mecanismul**. Procedural content generation este valoroasă pentru că produce varietate și scală fără a cere authoring manual pentru fiecare detaliu, iar survey-urile din domeniu separă conținutul pe straturi: spațiu, sisteme, scenarii, design și conținut derivat. Asta se potrivește perfect unui plugin Minecraft: poți genera procedural rute comerciale, povești locale, reacții la vreme, puncte de interes, reputații de sat și mini-evenimente.

12

Tehnicile cele mai eficiente pentru realism perceput sunt următoarele:

Tehnică	Implementare concretă în plugin	Efect perceput
Procedural generation	Seed per regiune, lot, facțiuni și sezon; șabloane + parametri	Lumea pare vastă și coerentă, nu aleatoare brut
Comportament stochastic	Utility scores cu jitter mic și cooldown-uri diferite	NPC-urile nu par „clone”
Informație întârziată sau parțială	Zvonuri locale, memorie expirabilă, percepție cu rază și obstacole	Elimină omnisciența artificială
Sensory cues	Particles localizate, sunete pe categorii, lumină sau afișaje contextuale	Jucătorul „simte” sistemul înainte să-l citească
Delay-uri plauzibile	Timp de gândire, drum, producție, încărcare, somn, tranzacție	Reduce impresia de teleport logic
Randomness controlat	Seed-uri per eveniment + log de cauzalitate	Debugging posibil fără a pierde varietatea
Client-side effects	Efecte vizuale/audio și texturi mentale, fără logică server grea	Mare impact perceptiv, cost mic

Paper recomandă ParticleBuilder pentru particles, iar Spigot permite redarea sunetelor pe categorii și chiar control de variație prin seed. Acest lucru e foarte util: dacă trimiți un sunet sau particle când agentul „observă” ceva, jucătorul va interpreta natural că agentul are percepție, chiar dacă logica internă este simplă.

13

Un principiu foarte important este **„percepția primează asupra simulării perfecte”**. Pe un VPS modest, rezultate excelente apar dacă păstrezi modelul intern relativ simplu, dar sincronizezi bine micro-indicii: privirea agentului, oprirea înainte de decizie, particule subtile la observație, întârziere la răspuns, schimbarea tonului audio într-o zonă cu pericol, mici deviații de rută și memorie locală a locurilor vizitate. Acesta este genul de realism „greu de clonat”, fiindcă vine din compoziția mai multor sisteme simple.

Arhitectură și fluxuri de date

Arhitectura recomandată este una **event-driven, actor-like, cu cozi și bugete**. Agenții nu apelează direct LLM-ul. Ei emit intenții sau cereri către un orchestrator AI care decide dacă răspunsul poate veni din cache, din euristici locale sau dintr-un provider extern. Persistența, profiling-ul și rate limiting-ul trebuie să fie servicii de infrastructură, nu cod împrăștiat prin listeners. Paper documentează explicit scheduling-ul, PDC, databases și suportul Paper/Folia; pentru standard Paper folosești scheduler-ul Bukkit/main thread pentru world updates, iar pentru Folia există global, region, async și entity schedulers. ¹⁴

```
flowchart LR
    A[Player events / World events] --> B[Scenario Engine]
    B --> C[Agent Simulation Engine]
    C --> D[Perception]
    C --> E[Utility AI / Behavior Trees]
    E --> F[Action Queue]
    F --> G[Main-thread World Mutations]

    E --> H[AI Orchestrator]
    H --> I[Heuristics / Templates]
    H --> J[Prompt Cache]
    H --> K[Budget & Rate Limiter]
    K --> L[External AI Provider]
    K --> M[Local Sidecar AI]

    C --> N[Economy Engine]
    C --> O[Weather / Hazard Engine]
    C --> P[Relationship & Reputation]

    C --> Q[Persistence Service]
    Q --> R[(SQLite/Postgres)]
    H --> Q
    N --> Q
    O --> Q
    P --> Q
```

Modelul de date merită să fie explicit și testabil. Dacă îl păstrezi doar în obiecte Bukkit, vei sfârși prin a amesteca lumea, starea și infrastructura într-un mod greu de profile-uit și refactorizat. Paper are PDC pentru date locale mici atașate entităților sau itemelor, dar pentru stări bogate și interogabile merită o bază de date. Paper recomandă baze de date pentru volume mai mari și explică diferența dintre baze file-based și standalone. ¹⁵

```
erDiagram
    AGENT ||--o{ MEMORY : has
    AGENT ||--o{ GOAL : pursues
    AGENT ||--o{ INVENTORY_SLOT : owns
```

```

AGENT }o--|| HOUSEHOLD : belongs_to
AGENT }o--o{ RELATIONSHIP : participates_in
AGENT }o--o{ JOB_ASSIGNMENT : works_in

HOUSEHOLD ||--o{ STOCKPILE : stores
MARKET ||--o{ PRICE_TICK : records
MARKET ||--o{ TRADE_ORDER : clears
REGION ||--o{ MARKET : contains
REGION ||--o{ WEATHER_STATE : experiences
REGION ||--o{ RESOURCE_NODE : contains
SCENARIO ||--o{ EVENT_INSTANCE : emits
EVENT_INSTANCE }o--o{ AGENT : affects

```

Fluxul ideal al unei decizii AI este asincron, dar mutațiile lumii revin pe thread-ul sigur pentru Paper. Asta urmează direct recomandările Paper despre task-uri asincrone și pericolele accesului la world state din async. ¹⁶

```

sequenceDiagram
    participant Tick as Tick Loop
    participant Agent as AgentController
    participant Plan as Planner
    participant Cache as Prompt Cache
    participant AI as AI Provider
    participant Main as Main Thread

    Tick->>Agent: sense()
    Agent->>Plan: requestPlan(snapshot)
    Plan->>Cache: lookup(hash(snapshot))
    alt cache hit
        Cache-->>Plan: plan
    else cache miss
        Plan->>AI: async request
        AI-->>Plan: compact plan
        Plan->>Cache: store
    end
    end
    Plan-->>Agent: future(plan)
    Agent->>Main: applyAction() on safe scheduler

```

Pe un VPS modest, recomand practic trei bucle:

1. **Low-level actuation** la 20 tps sau la tick-ul standard al jocului, dar doar pentru mutații mici.
2. **High-level cognition** la 2-5 Hz pe agenți activi din jurul jucătorilor.
3. **Strategic world simulation** la 0.2-1 Hz pe regiuni sau piețe, nu pe fiecare agent.

Aceasta este o recomandare de proiectare, nu o limită impusă de API, dar este aliniată cu realitatea Paper: 20 tps înseamnă 50 ms per tick, iar task-urile lente pe main thread provoacă lag. ⁶

Alegerea AI-ului și a bibliotecilor Java

Pentru apeluri externe, alege providerul în funcție de **rolul exact** al AI-ului în plugin, nu după hype. Dacă vrei etichete, rutare, scorare sau scurte planuri structurale, modelul cel mai ieftin și mai rapid este aproape întotdeauna suficient. Dacă vrei rezumate narative, explicații sau dialoguri „de facțiuni”, ridici nivelul modelului doar pe acele rute. Documentațiile furnizorilor consultate diferă semnificativ la preț, throughput expus oficial, free tier și control operațional. ¹⁷

Furnizor	Model recomandat	Cost standard	Semnal oficial de latență / throughput	Free tier / trial	Potrivire pentru plugin	Surse
Google ¹⁸	Gemini 2.5 Flash-Lite	\$0.10 input / \$0.40 output per 1M tokens	Google îl descrie ca „fastest performance” pentru task-uri high-frequency și extremely low-latency; suportă Batch, Caching, Flex și Priority; în Tier 1 are 10M batch enqueued tokens pentru Flash-Lite	Da, free tier explicit în pricing	Cea mai bună alegere pentru scorare, clasificare, sumarizare scurtă, world events și fallback generic ieftin	¹⁹
Groq ²⁰	Llama-3.1-8b-instant sau GPT OSS 20B	\$0.05 / \$0.08 per 1M tokens pentru Llama 3.1 8B; \$0.075 / \$0.30 pentru GPT OSS 20B	Groq publică viteze oficiale: 560 t/s pentru Llama 3.1 8B și 1000 t/s pentru GPT OSS 20B; are și headers explicite de rate limit	Există Free tier; pentru Llama 3.1 8B, în free plan apar 30 RPM și 6K TPM; Developer plan ridică la 1K RPM și 250K TPM	Excelent când vrei latență minimă și modele open compatibile	²¹

Furnizor	Model recomandat	Cost standard	Semnal oficial de latență / throughput	Free tier / trial	Potrivire pentru plugin	Surse
OpenAI 22	GPT-5.4 mini	\$0.75 / \$4.50 per 1M tokens standard; Priority: \$1.50 / \$9.00	OpenAI îl etichetează „Faster”; 400K context, tool use, file search, computer use; rate limits și bugete sunt configurabile la nivel de proiect / usage tier	Nu am găsit în sursele consultate un free tier API general public	Bun când vrei integrare matură, output mai stabil și funcții bogate de tool use	23
Anthropic 24	Claude Haiku 4.5	\$1 / \$5 per MTok	Anthropic îl numește „fastest, most cost-efficient model”; pagina modelului afirmă și 4–5x mai rapid decât Sonnet 4.5; prompt caching reduce costul și timpul pentru prefixe repetitive	Free tier API neprecizat în sursele consultate	Foarte bun pentru bucle agentice scurte, dialog și caching de context lung	25
Mistral AI 26	Mistral Small 4	\$0.15 / \$0.60 per 1M tokens	256K context; model card-ul îl poziționează ca model mic puternic pentru instruct/reasoning/coding; rate limits la nivel de organizație și tier-uri automate	Da, Experiment plan gratuit cu rate limits conservatoare	Foarte bun pentru rezumate, planificare și scenarii narative scurte la cost mic	27

Furnizor	Model recomandat	Cost standard	Semnal oficial de latență / throughput	Free tier / trial	Potrivire pentru plugin	Surse
Cloudflare ²⁸	Workers AI pe modele open mici/medii	\$0.011 / 1000 Neurons, cu 10,000 Neurons/zi gratuite	Workers AI rulează la edge; rate limits implicite pentru text generation: 300 RPM, unele modele au limite mai mari	Da, 10,000 Neurons/zi	Bun dacă pluginul tău comunică cu o infrastructură edge proprie; comparația per-token nu este directă	²⁹

Observația-cheie este că **Gemini 2.5 Flash-Lite și Groq** au cel mai puternic profil „ieftin + rapid” în sursele oficiale consultate, dar în moduri diferite: Gemini oferă free tier și ecosistem bogat de instrumente, în timp ce Groq oferă cele mai explicite cifre publice de viteză și rate-limit headers foarte utile în operare. OpenAI și Anthropic sunt opțiuni excelente dacă ai nevoie de output mai robust și de ecosisteme de tool use / caching mature, însă la cost mai mare per output token. ³⁰

Pentru self-hosting și low-budget, opțiunile principale sunt acestea:

Motor self-hosted	Model de operare	Avantaj în plugin	Limitare principală	Surse
Ollama	Sidecar local pe HTTP, implicit pe localhost: 11434	Cel mai simplu de pornit; API stabil; expune metrice precum total_duration și eval_duration; compatibilitate cu OpenAI Responses pentru unele fluxuri	Pe CPU/VPS modest devine repede insuficient pentru roluri „centrale”	³¹
vLLM	Engine de serving cu PagedAttention și API compatibil OpenAI	Throughput mare pe GPU; paper-ul raportează 2–4× throughput la aceeași latență vs sisteme anterioare	Are sens mai ales dacă ai GPU real și workload susținut	³²
llama.cpp server	Server HTTP lightweight în C/C++	Foarte bun pentru footprint mic, modele cuantizate, CPU/GPU mix, continuous batching și compatibilitate OpenAI/Anthropic	Calitatea și viteza depind masiv de cuantizare și hardware	³³

Motor self-hosted	Model de operare	Avantaj în plugin	Limitare principală	Surse
LocalAI	Binary/container local, compatibil OpenAI/Anthropic/Open Responses	Privacy first, API drop-in, mai multe tipuri de modele și agenți	Ecosistem mai eterogen; performanța depinde de backend	34

Concluzia practică pentru un VPS modest este simplă: dacă nu ai GPU dedicat, self-hosting-ul e bun mai ales pentru **clasificare, embeddings, safety checks, șabloane și fallback**, nu pentru „mintea principală” a lumii. Dacă ai GPU, vLLM e opțiunea serioasă pentru serving, iar pentru dezvoltare locală și testare rapidă, Ollama și llama.cpp sunt mai simple. Aceasta este o recomandare de inginerie bazată pe capabilitățile documentate și pe costul operațional implicit al acestor stack-uri. 35

Pentru bibliotecile Java/Spigot/Paper, recomandarea mea este:

Bibliotecă / API	Rol	Ce oferă	Verdict pentru acest proiect	Surse
JDK <code>HttpClient</code>	HTTP	API standard JDK, HTTP/1.1 și HTTP/2, client imutabil, reutilizabil	Alegerea implicită dacă vrei zero dependențe	36
OkHttp	HTTP	Eficient implicit, retry/recovery, TLS modern, certificate pinning, connection pooling partajat	Mai bun dacă ai multe apeluri externe și vrei ergonomie mai bună	37
Jackson Databind	JSON	Data binding și tree model bogat	Cel mai bun pentru payload-uri complexe și DTO-uri stabile	38
Gson	JSON	Simplu, ușor, obiect↔JSON	Bun la payload-uri mici; mai puțin expresiv decât Jackson	39
<code>CompletableFuture</code>	Async orchestration	<code>CompletionStage</code> , chaining, <code>allOf</code> , timeout-uri, executors expliți	Foarte potrivit pentru pipeline-uri AI+cache+fallback	40

Bibliotecă / API	Rol	Ce oferă	Verdict pentru acest proiect	Surse
<code>ScheduledExecutorService</code>	Scheduler independent	Task-uri periodice sau întârziate în afara tick-ului serverului	Bun pentru polling de cozi, expirări și retry; nu pentru mutații de world state	41
HikariCP	Pooling JDBC	Pool ușor, production-ready	Recomandat pentru MySQL/Postgres; Paper îl arată în ghidul de DB, dar nu îl bundle-uește	42
SQLite prin JDBC	Persistență locală	Simplu, file-based; driverul e inclus în Paper	Excelent pentru MVP și servere mici	2
ONNX Runtime Java	Inferență locală pe model ONNX	Binding Java pentru JVM; artefacte CPU și GPU în Maven Central	Cea mai bună opțiune pentru clasificatori și modele mici în-JVM	43
DJL	Abstracție ML Java	Experiență nativă Java, engine abstraction, inclusiv ONNX Runtime engine	Bun dacă vrei să schimbi backend-uri fără a rescrie mult cod	44
TensorFlow Java	Inferență ML	API Java existent, dar API-ul <code>org.tensorflow</code> documentat este marcat deprecated	Evită-l pentru proiect nou, dacă nu ai un motiv clar	45

Plan de dezvoltare, testare și operare

Ordinea de dezvoltare contează mai mult decât sofisticarea AI-ului. Dacă începi cu provideri multipli, dialog generativ și economii complexe înainte să ai metrici, vei avea o simulare imposibil de diagnosticat. În schimb, construiește straturi verificabile. Paper are ghiduri separate pentru project setup, scheduling, PDC, databases și profiling, iar comanda recomandată pentru performanță este `/spark`; în JVM, JFR are overhead tipic sub 1% pe configurația implicită, iar async-profiler este un profiler de sampling cu overhead redus. 46

O ordine sănătoasă de milestones este aceasta:

Milestone	Livrabil	Criteriu de acceptare
M0	Schelet plugin, config, servicii, persisted state	Plugin pornește, salvează, încarcă, expune metrice
M1	Agent local fără AI extern	NPC merge, observă, alege între 3–5 acțiuni, persistă
M2	Economie + vreme	Prețuri și rutine se schimbă după resurse și vreme
M3	Scenariu open-world	Sat funcțional 30–50 agenți cu tick budget stabil
M4	Orchestrator AI + cache	1 provider extern + cache + timeout + fallback
M5	Scenariu scripted disaster	Eveniment multi-fază cu reacții, evacuare, refacere
M6	Tuning și profilare	Lag controlat, bugete AI respectate, retry/rate limits
M7	Scalare Paper/Folia	Compatibilitate explicită și politici per scheduler

Strategiile de testare ar trebui împărțite în patru clase. Prima: **unit tests pure Java** pentru economie, utilități, scoring, memorie și planificare. A doua: **simulation replay tests** bazate pe seed fix și golden traces, ca să poți reproduce bug-urile într-o lume stochastică. A treia: **integration tests** care pornesc un server Paper real și rulează scenarii scurte de 1–5 minute. A patra: **soak tests** de 30–120 minute pe hartă mică, cu spawn controlat și metrice de AI cache hit, p99 response time, queue depth, tick cost și write latency în DB. Recomandarea de a ancora testele în seed-uri vine direct din faptul că Paper măsoară munca în tick-uri, iar latența percepută și drift-ul logic se văd în timp, nu doar în apeluri izolate. ⁴⁷

Pentru profiling, ordinea practică este:

1. `/spark` pentru hotspot-uri de tick și plugin-level attribution.
2. JFR pentru GC, IO, lock contention și pattern-uri JVM pe perioade mai lungi.
3. async-profiler pentru flame graphs CPU/allocation când un subsystem devine suspect.

Aceste instrumente sunt complementare, nu concurente. Paper însuși recomandă `/spark` drept instrumentul pe care să te bazezi pentru informații de performanță ale serverului, iar JFR și async-profiler sunt potrivite pentru nivel JVM. ⁴⁸

În exploatare, cea mai importantă regulă este **bugetarea**. Setează bugete maxime per tick, per regiune, per agent activ și per minut de API extern. Dacă bugetul este depășit, sistemul trebuie să degradeze grațios: scoate AI-ul extern, folosește planuri cached, reduce frecvența de replanificare, înlocuiește textul cu template-uri locale și suspendă actualizările agenților foarte îndepărtați de jucători.

Exemple de integrare

Mai jos este un exemplu minimal pentru un gateway HTTP bazat pe JDK `HttpClient`. JDK 21 oferă `HttpClient` standard pentru trimitere de request-uri și răspunsuri, iar `CompletableFuture` este potrivit pentru chaining și timeouts. În jurul lui, trebuie să construiești rate limiting, timeout-uri și cache.

49

```
public final class AiGateway {
    private final HttpClient http = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_2)
        .connectTimeout(Duration.ofSeconds(3))
        .build();

    private final ObjectMapper json = new ObjectMapper();
    private final URI endpoint;
    private final String apiKey;

    public AiGateway(URI endpoint, String apiKey) {
        this.endpoint = endpoint;
        this.apiKey = apiKey;
    }

    public CompletableFuture<AiPlan> requestPlanAsync(WorldSnapshot snapshot) {
        try {
            Map<String, Object> body = Map.of(
                "model", "gemini-2.5-flash-lite",
                "contents", List.of(Map.of(
                    "role", "user",
                    "parts", List.of(Map.of(
                        "text", snapshot.toCompactPrompt()
                    ))
                ))
            );

            HttpRequest request = HttpRequest.newBuilder(endpoint)
                .timeout(Duration.ofSeconds(5))
                .header("Content-Type", "application/json")
                .header("Authorization", "Bearer " + apiKey)
                .POST(HttpRequest.BodyPublishers.ofString(json.writeValueAsString(body)))
                .build();

            return http.sendAsync(request, HttpResponse.BodyHandlers.ofString())
                .thenCompose(resp -> {
                    if (resp.statusCode() == 429) {
                        return CompletableFuture.failedFuture(
                            new IllegalStateException("Rate limited");
                        );
                    }
                });
        } catch (Exception e) {
            return CompletableFuture.failedFuture(e);
        }
    }
}
```

```

        }
        if (resp.statusCode() / 100 != 2) {
            return CompletableFuture.failedFuture(
                new IllegalStateException("HTTP " +
resp.statusCode()));
        }
        try {
            JsonNode root = json.readTree(resp.body());
            String text = extractText(root); // normalizează
provider-specific
            return
CompletableFuture.completedFuture(AiPlan.parse(text));
        } catch (Exception e) {
            return CompletableFuture.failedFuture(e);
        }
    })
    .orTimeout(6, TimeUnit.SECONDS);
} catch (Exception e) {
    return CompletableFuture.failedFuture(e);
}
}

private String extractText(JsonNode root) {
    // provider-specific normalization
    return root.path("candidates").get(0)
        .path("content").path("parts").get(0)
        .path("text").asText();
}
}

```

Exemplul următor arată modelul corect pentru Paper: IO și inferența rămân async; mutațiile lumii se întorc pe scheduler-ul sigur. Paper avertizează explicit că metodelor Bukkit care citesc sau schimbă world state nu le priește async. ¹⁶

```

public final class AgentPlannerService {
    private final JavaPlugin plugin;
    private final AiGateway aiGateway;
    private final PlanCache cache;

    public AgentPlannerService(JavaPlugin plugin, AiGateway aiGateway,
PlanCache cache) {
        this.plugin = plugin;
        this.aiGateway = aiGateway;
        this.cache = cache;
    }
}

```

```

public void maybeReplan(NpcAgent agent) {
    if (!agent.shouldReplanNow()) return;

    WorldSnapshot snapshot = WorldSnapshot.from(agent);
    String cacheKey = snapshot.hash();

    AiPlan cached = cache.get(cacheKey);
    if (cached != null) {
        applyOnMainThread(agent, cached);
        return;
    }

    aiGateway.requestPlanAsync(snapshot)
        .exceptionally(ex -> fallbackPlan(snapshot, ex))
        .thenAccept(plan -> {
            cache.put(cacheKey, plan, Duration.ofMinutes(10));
            applyOnMainThread(agent, plan);
        });
}

private AiPlan fallbackPlan(WorldSnapshot snapshot, Throwable ex) {
    return AiPlan.localHeuristic(snapshot);
}

private void applyOnMainThread(NpcAgent agent, AiPlan plan) {
    plugin.getServer().getScheduler().runTask(plugin, () -> {
        if (!agent.isStillValid()) return;
        agent.applyPlan(plan);
    });
}
}

```

Controller-ul agentului ar trebui să țină managerul de planuri separat de stratul de acțiune. Asta permite să schimbi providerul, să activezi cache sau să oprești complet AI-ul extern fără să distrugi gameplay-ul de bază.

```

public final class NpcAgentController implements Runnable {
    private final UUID entityId;
    private final AgentPlannerService planner;
    private int stuckTicks = 0;

    public NpcAgentController(UUID entityId, AgentPlannerService planner) {
        this.entityId = entityId;
        this.planner = planner;
    }
}

```

```

@Override
public void run() {
    Entity entity = Bukkit.getEntity(entityId);
    if (!(entity instanceof LivingEntity living)) return;

    NpcAgent agent = NpcRegistry.get(entityId);
    if (agent == null) return;

    agent.sense(living);

    if (agent.isPathBlocked()) {
        stuckTicks++;
    } else {
        stuckTicks = 0;
    }

    if (stuckTicks > 40) {
        agent.recoverPath(); // repath / sidestep / cancel action
        stuckTicks = 0;
    }

    if (agent.hasImmediateAction()) {
        agent.executeImmediateAction(living);
    } else {
        planner.maybeReplan(agent);
    }
}
}

```

Dacă vrei să menții realismul la cost mic, cel mai eficient pattern este acesta:

- planurile lungi sunt generate rar și pe snapshot-uri compacte;
- dialogul și textul sunt template-first, AI-second;
- reputația, relațiile și economia sunt locale și deterministe;
- AI-ul extern completează „ce nu merită authoring manual”, nu „ce trebuie să fie stabil și rapid”.

Aceasta este, în practică, strategia care maximizează impresia de lume inteligentă la cost și latență rezonabile.

Securitate, confidențialitate și întrebări deschise

Pentru securitate și privacy, minimul profesionist este: **chei în environment variables sau secret vault**, chei separate pe medii, bugete per proiect/workspace, logging minimizat, prompt-uri fără date sensibile brute și degradare sigură la 429/5xx. Google recomandă explicit folosirea environment variables pentru chei și avertizează că hardcoding-ul e nesigur; Mistral spune că cheia completă este afișată o singură dată și nu trebuie pusă în version control; Anthropic și Groq oferă workspaces/projects pentru segmentare și control al cheilor și costurilor; OpenAI oferă limite și bugete configurabile per proiect. ⁵⁰

Pentru confidențialitate, furnizorii diferă. În sursele oficiale consultate, Mistral afirmă că datele trimise prin API nu sunt folosite pentru antrenare; OpenAI spune că nu își antrenează modelele pe datele organizației by default; Groq oferă Zero Data Retention; Anthropic notează că prompt caching este eligibil pentru ZDR; LocalAI și stack-urile self-hosted păstrează datele locale prin design. Dacă pluginul procesează chat de jucători, nume reale, IP-uri sau loguri administrative, aceste diferențe contează mult și trebuie proiectate explicit, nu „moștenite” accidental din providerul ales. ⁵¹

Pentru rate limits, tratează 429 ca eveniment normal de sistem, nu ca excepție „rară”. Groq documentează `retry-after` și header-ele `x-ratelimit-*`; OpenAI recomandă exponential backoff și explică faptul că limitele pot fi cuantizate pe ferestre scurte; Anthropic și Gemini explică limite pe organizație/proiect și pe dimensiuni precum RPM, TPM și RPD. În plus, Groq, Mistral și Anthropic spun clar că limitele se aplică la nivel de organizație/proiect/workspace, nu doar la cheia individuală. ⁵²

Există și o limitare importantă legată de siguranță: ieșirile AI pot fi greșite, iar politicile Groq subliniază explicit că output-urile trebuie evaluate și validate, mai ales dacă influențează decizii cu impact. Într-un plugin Minecraft asta se traduce printr-o regulă simplă: AI-ul poate sugera, prioriza sau colora un eveniment, dar nu ar trebui să execute direct acțiuni administrative ireversibile, sancțiuni sau modificări destructive fără validări locale. ⁵³

Întrebările deschise cele mai importante, pe care sursele consultate nu le fixează singure, sunt acestea:

- dacă VPS-ul are sau nu GPU, pentru că asta schimbă complet viabilitatea self-hosting-ului;
- câți jucători simultani și câți agenți activi vrei în jurul fiecărui jucător;
- cât de „canonică” trebuie să fie lumea între restart-uri și upgrade-uri;
- câtă narațiune textuală vrei față de simulare pur sistemică;
- dacă ținta reală este Paper standard sau compatibilitate Paper/Folia din prima zi.

Dacă aceste constrângeri rămân nedefinite, recomandarea mea finală este fermă: construiește un plugin **Paper-first, AI-hybrid, cache-heavy, deterministic-by-default, stochastic-at-the-edges**, cu un singur provider extern activ în MVP și cu self-hosted doar ca fallback sau mediu de test. Aceasta oferă cel mai bun raport între realism perceput, cost, latență, debuggability și șanse reale de a livra ceva care funcționează pe un VPS modest.

¹ ⁵ ¹⁵ ²⁶ ²⁸ ⁴⁶ <https://docs.papermc.io/paper/dev/api/>
<https://docs.papermc.io/paper/dev/api/>

² ²² [Using databases | PaperMC Docs](https://docs.papermc.io/paper/dev/using-databases)
<https://docs.papermc.io/paper/dev/using-databases>

³ ¹⁷ ²⁰ ²¹ <https://console.groq.com/docs/models>
<https://console.groq.com/docs/models>

⁴ ⁹ <https://faculty.cc.gatech.edu/~riedl/pubs/tidse06a.pdf>
<https://faculty.cc.gatech.edu/~riedl/pubs/tidse06a.pdf>

⁶ ¹⁴ ¹⁶ ⁴⁷ [Scheduling | PaperMC Docs](https://docs.papermc.io/paper/dev/scheduler)
<https://docs.papermc.io/paper/dev/scheduler>

- 7 https://pages.cs.wisc.edu/~dyer/cs540/notes/06_gamePlaying.pdf
https://pages.cs.wisc.edu/~dyer/cs540/notes/06_gamePlaying.pdf
- 8 13 <https://docs.papermc.io/paper/dev/particles/>
<https://docs.papermc.io/paper/dev/particles/>
- 10 11 18 <https://arxiv.org/abs/2005.05842>
<https://arxiv.org/abs/2005.05842>
- 12 <https://dl.acm.org/doi/abs/10.1145/2422956.2422957>
<https://dl.acm.org/doi/abs/10.1145/2422956.2422957>
- 19 30 <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash-lite>
<https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash-lite>
- 23 <https://developers.openai.com/api/docs/models>
<https://developers.openai.com/api/docs/models>
- 24 50 <https://ai.google.dev/gemini-api/docs/api-key>
<https://ai.google.dev/gemini-api/docs/api-key>
- 25 <https://www.anthropic.com/pricing>
<https://www.anthropic.com/pricing>
- 27 <https://docs.mistral.ai/models/model-cards/mistral-small-4-0-26-03>
<https://docs.mistral.ai/models/model-cards/mistral-small-4-0-26-03>
- 29 <https://developers.cloudflare.com/workers-ai/platform/pricing/>
<https://developers.cloudflare.com/workers-ai/platform/pricing/>
- 31 35 <https://docs.ollama.com/api/introduction>
<https://docs.ollama.com/api/introduction>
- 32 vLLM
https://vllm.ai/?utm_source=chatgpt.com
- 33 <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
<https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- 34 <https://localai.io/docs/overview/index.html>
<https://localai.io/docs/overview/index.html>
- 36 49 HttpClient (Java SE 21 & JDK 21)
https://docs.oracle.com/en/java/javase/21/docs/api/java.net.http/java/net/http/HttpClient.html?utm_source=chatgpt.com
- 37 Overview - OkHttp
https://square.github.io/okhttp/?utm_source=chatgpt.com
- 38 FasterXML/jackson-databind: General data-binding ...
https://github.com/FasterXML/jackson-databind?utm_source=chatgpt.com
- 39 Gson User Guide
https://google.github.io/gson/UserGuide.html?utm_source=chatgpt.com
- 40 <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CompletableFuture.html>
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CompletableFuture.html>

- 41 <https://cr.openjdk.org/~jlaskey/templates/docs/api/java.base/java/util/concurrent/ScheduledExecutorService.html>
<https://cr.openjdk.org/~jlaskey/templates/docs/api/java.base/java/util/concurrent/ScheduledExecutorService.html>
- 42 <https://github.com/brettwooldridge/hikaricp>
<https://github.com/brettwooldridge/hikaricp>
- 43 <https://onnxruntime.ai/docs/get-started/with-java.html>
<https://onnxruntime.ai/docs/get-started/with-java.html>
- 44 **Deep Java Library: Main**
https://docs.djl.ai/?utm_source=chatgpt.com
- 45 **org.tensorflow | Java**
https://www.tensorflow.org/api_docs/java/org/tensorflow/package-summary?utm_source=chatgpt.com
- 48 **Commands**
https://docs.papermc.io/paper/reference/commands/?utm_source=chatgpt.com
- 51 <https://docs.mistral.ai/admin/security-access/privacy>
<https://docs.mistral.ai/admin/security-access/privacy>
- 52 <https://console.groq.com/docs/rate-limits>
<https://console.groq.com/docs/rate-limits>
- 53 <https://console.groq.com/docs/legal/ai-policy>
<https://console.groq.com/docs/legal/ai-policy>